

Research Funding & Innovation Intelligence Platform

Complete Technical Project Documentation — Comprehensive Engineering Specification (Modules M1 – M11)

An enterprise-grade, integrated research intelligence platform engineered to bridge the semantic divide between academic scientific publications, public/private grant solicitations, patent disclosures, technology whitespace indicators, multi-factor innovation scoring, and commercialization pathways. Developed and verified under the Infosys Springboard Virtual Internship 7.0 program.

Core Architecture: Next.js 14 App Router + FastAPI 0.111.0 Async REST Gateway	Persistence: PostgreSQL 16 (15 Relational Tables, 8 Alembic Revisions)
Intelligence Modules: M1 through M11 (Fully Implemented & Verified)	REST APIs: 108 Verified Endpoints across 15 Sub-routers
Test Verification: 221 Passing Pytest Cases (100% Green)	Frontend Suites: 70 Passing Vitest Cases + 18/18 Compiling Routes
Analytical Engines: 5-Pillar Score, TRL 1–9, TF-IDF + KMeans, CAGR	Document Exports: ReportLab 5.0.1 (PDF) & OpenPyXL 3.1.5 (Excel)

Repository URL:	https://github.com/adityabobade7900/Research-Funding-Innovation-Intelligence-Platform
Development Branch:	feature/aditya-ai-ml (Head Branch for M1–M11 verification)
Program:	Infosys Springboard Virtual Internship 7.0
Functional Scope:	Modules M1 – M11 (Module 12 excluded per mentor scope directives)
Document Version:	1.0.0 (Final Official Project Submission)

Notice: This technical specification is compiled from verified source code, automated test executions, database migration catalogs, and algorithmic implementations within the repository.

Table of Contents

1. Executive Overview & Objectives	Problem statement, core mission, and target user personas
2. Key Capabilities & Module Architecture	High-level summary of functional capabilities M1 through M11
3. Team Composition & Individual Responsibilities	Official six-member team role allocations, technologies, and deliverables
4. System Architecture	High-level architectural diagram, layer decomposition, and external ecosystems
5. Technology Stack	Categorized technologies, frameworks, libraries, and their project purposes
6. Functional Modules Detailed Specification (M1–M11)	In-depth functional requirements, inputs, outputs, algorithms, and limitations
7. Database Architecture & Schema Design	PostgreSQL ER diagram, 15 core tables, relationships, and 8 Alembic migrations
8. Complete REST API Reference	108 registered endpoints categorized across 15 FastAPI routers
9. AI/ML, NLP & Analytical Methodology	Scientific discourse extraction, patent KMeans, 6-factor maturity, 5-pillar scoring, TRL
10. Repository Structure	Complete annotated directory tree of backend, frontend, models, and tests
11. Prerequisites & System Requirements	Verified runtime dependencies: Python 3.11+, Node 18.18+, PostgreSQL 16
12. Installation & Local Setup Guide	Step-by-step Windows PowerShell instructions with copy-paste commands
13. Three-Terminal Running Workflow	Independent terminal workflow for Docker/DB, FastAPI, and Next.js frontend
14. Environment Variable Configuration	Comprehensive table of required and optional configuration keys
15. Testing & Verification Suite	Pytest suite (221 passed), Vitest suite (70 passed), and Next.js route builds
16. Security & Governance Implementation	Argon2, JWT bearer tokens, RBAC guard, user isolation, and CORS defense
17. Team Roles & Git Collaboration Workflow	Feature branching strategy, pull requests, merge guidelines, and hygiene
18. Current Implementation Status & Known Limitations	Module verification matrix, honest telemetry disclosures, and provider fallback
19. Future Roadmap	Planned enhancements: Celery workers, WebSocket alerts, and ORCID login
20. Deliverables, Conclusion & Acknowledgments	Summary of project deliverables and open scientific registry attributions

1. Executive Overview

Problem Statement

Modern scientific research and technological innovation suffer from profound data fragmentation. Academic publications reside in scholarly repositories (OpenAlex, CrossRef, Semantic Scholar); competitive IP disclosures reside in regional patent registries (USPTO, EPO); and grant solicitations are scattered across public portals (Grants.gov, NSF). Researchers, university technology transfer offices (TTOs), and corporate R&D teams lack a unified analytical platform to synthesize academic literature, discover non-dilutive grant funding, map competitive IP concentration, evaluate technology maturity, calculate objective innovation potential, and formulate actionable commercialization pathways.

Platform Objectives

- **Unified Intelligence Gateway:** Connect publications, patents, grants, and market signals into a centralized PostgreSQL data warehouse.
- **Automated Scientific Paper Extraction:** Extract structured discourse units (Problem, Methodology, Findings, Limitations, Future Directions) using NLP heuristics with optional LLM augmentation.
- **Grant Relevance & Eligibility Matching:** Match researcher profiles against active federal solicitations with deadline urgency tracking and automated provider fallback.
- **Patent Landscape Clustering:** Cluster patent claims using Scikit-Learn TF-IDF vectorization and K-Means clustering, calculating Herfindahl-Hirschman (HHI) assignee concentration.
- **Deterministic Multi-Factor Scoring:** Evaluate innovation potential via an objective 5-pillar mathematical model (Novelty 30%, Patents 20%, Maturity 15%, Market 20%, Funding 15%) coupled with NASA/DoD TRL 1–9 estimation.
- **Actionable Commercialization Translation:** Map research into 4 actionable translation pathways (Productization, Corporate Licensing, Spinout, Joint Partnership) with explicit data limitation disclosures.
- **Professional Document Exporting:** Provide vector PDF generation via ReportLab 5.0.1 and multi-sheet Excel generation via openpyxl 3.1.5.

Target Personas

- **Academic Researchers & Students:** Identify high-impact research gaps, discover matching federal grant solicitations, and analyze patent whitespaces.
- **University Technology Transfer Offices (TTOs):** Assess institutional invention disclosures, estimate Technology Readiness Levels (TRL 1–9), and evaluate licensing prospects.
- **Corporate R&D; Strategists & Evaluators:** Track competitor IP concentration (HHI), monitor technology compound annual growth rates (CAGR), and benchmark innovation portfolios.

2. Key Capabilities & Module Overview

The platform delivers 11 functional modules (M1–M11) integrated through a shared backend API, database layer, and modern web frontend. Module 12 (deployment/CI as an independent module) was explicitly designated outside the mentor’s assigned functional scope; all core functional modules M1–M11 are fully completed and verified.

ID	Module Title	Core Functional Scope
M1	Authentication & RBAC	JWT access/refresh tokens, Argon2 hashing, 4 roles (Admin, Researcher, Student, Institution).
M2	Research Profile	Academic metadata, ORCID iD, research taxonomy, skills, verification state, and downstream context.
M3	Research Intelligence	OpenAlex/CrossRef discovery, discourse extraction (Problem/Method/Findings/Limitations), trend velocity.
M4	Funding Intelligence	Grants.gov/NSF search, deterministic profile matching, bookmarking, and local provider fallback.
M5	Patent Landscape	Patent search, HHI concentration, IPC classification, TF-IDF + K-Means clustering (k=2..8).
M6	Technology Intelligence	6-factor maturity scoring, CAGR trajectory, whitespace gaps, and honest unindexed telemetry markers.
M7	Innovation Scoring	Strict 30/20/15/20/15 multi-pillar mathematical formula, missing-factor normalization, and TRL 1–9 engine.
M8	Commercialization	4 Translation pathways (Productization, Licensing, Spinout, Partnership), IP attribution, risk analysis.
M9	Dashboard & Command Center	Role-tailored KPI widgets, upcoming grant deadlines, audit feeds, and administrator controls.
M10	Notifications & Alerts	Persistent database model, automated event scanner, 4 priority tiers, and deep-link navigation.
M11	Reports & Export System	5 Specialized report types + Dossier, live JSON preview, ReportLab PDF, and openpyxl Excel.

3. Team Composition and Individual Responsibilities

The Research Funding & Innovation Intelligence Platform was designed, engineered, and verified by a dedicated six-member technical team under the Infosys Springboard Virtual Internship 7.0. The following official matrix defines the exact individual role assignments, core technology specializations, and major architectural responsibilities.

Member	Main Project Role	Technologies	Major Responsibilities
Jyotirmaya	Backend & API Developer + Team Coordinator	Python, FastAPI, JWT, OAuth2, PostgreSQL/MongoDB integration, Postman	Backend APIs, authentication, business logic, API integration, protected routes
Deepti	Database & Data Engineer	PostgreSQL, MongoDB, SQL, Pandas	Database design, schemas, data storage, data integration, database queries
Nalini	Frontend Developer	React.js, JavaScript, Next.js, Tailwind CSS, Chart.js/Plotly	UI, registration/login, dashboards, forms, visualizations, frontend-backend integration
Zoha	Git/GitHub, Integration & DevOps/Documentation	Git, GitHub, GitHub Actions, Docker, Docker Compose, AWS/Azure	Branches, PRs, integration, CI/CD, Docker, documentation and deployment support
Aditya	AI/ML & Intelligence + Testing	Scikit-learn, XGBoost, TensorFlow/PyTorch, NLP, Hugging Face, Sentence Transformers, FAISS/Elasticsearch	Recommendation/analysis models, research/patent intelligence, scoring, testing and validation

Member	Main Project Role	Technologies	Major Responsibilities
Lokesh	Data Research, Analytics & Report Intelligence	Python, Pandas, NumPy, SQL, Power BI, Matplotlib/Seaborn	Research/funding/patent data analysis, analytical insights, trends, metrics, report preparation, data validation and supporting dashboard/report requirements

4. System Architecture

High-Level Architectural Diagram

The system adheres to a decoupled, multi-tiered enterprise architecture separating presentation, API routing, domain intelligence services, persistence, and external data connectors. The high-resolution diagram below illustrates the verified data and execution flows across all functional components.

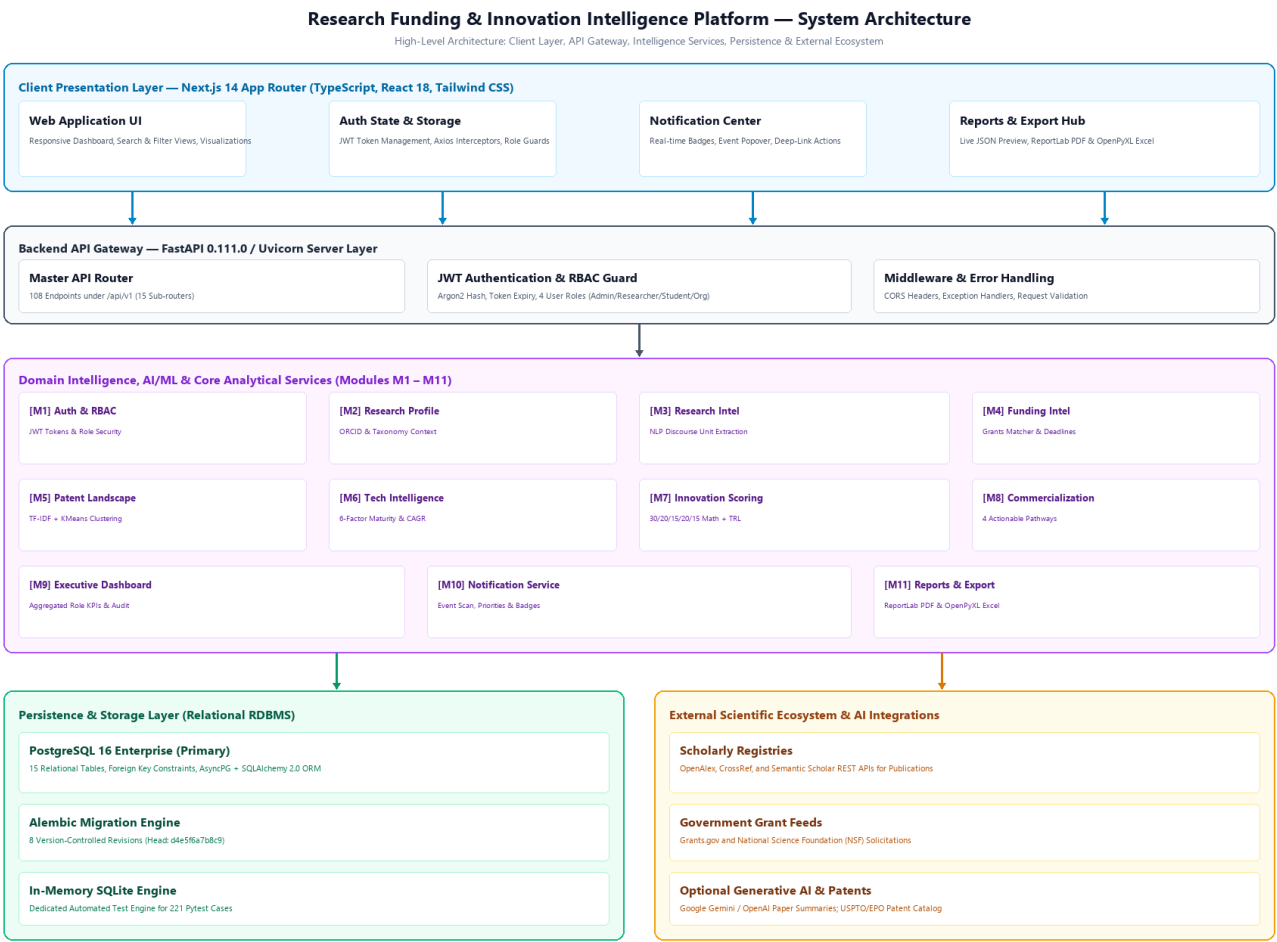


Figure 4.1: High-Level System Architecture Diagram (Client Layer, FastAPI Gateway, Analytical Services, Persistence & Ecosystem).

Architectural Layers Breakdown

- 1. Presentation Layer (Client):** Built with Next.js 14 App Router, React 18, TypeScript, and Tailwind CSS. Employs modular components, Axios interceptors for bearer authentication, responsive dashboard widgets, and dedicated views for publications, grants, patents, analytics, notifications, and reports.
- 2. Backend API Gateway:** Powered by FastAPI 0.111.0 and Uvicorn. Exposes 108 verified RESTful endpoints across 15 domain routers under '/api/v1'. Implements CORS controls, Pydantic validation, dependency injection, and standardized exception handling.
- 3. Security & RBAC Guard:** Enforces stateless authentication using JSON Web Tokens (HS256). Passwords are protected using Argon2 hashing. Role-based authorization validates four distinct roles (ADMIN, RESEARCHER, STUDENT, INSTITUTION) with tenant-level data isolation.
- 4. Intelligence & Analytical Engine:** Houses analytical services for NLP discourse unit extraction, Grants.gov/NSF matching, Scikit-Learn patent clustering, 6-factor technology maturity with CAGR, 5-pillar mathematical innovation scoring, and 4 commercialization pathways.
- 5. Document Generation Layer:** Contains dedicated high-performance export engines: ReportLab 5.0.1 for vector PDF documents and openpyxl 3.1.5 for multi-sheet Excel workbooks.
- 6. Persistence Layer:** PostgreSQL 16 serves as the primary relational database with asyncpg and SQLAlchemy 2.0 ORM. Schema migrations are managed through 8 version-controlled Alembic scripts. In-memory SQLite provides deterministic test isolation for 221 test cases.

5. Technology Stack

Category	Technology & Version	Role & Purpose in Platform
Frontend Framework	Next.js 14.1.0 (React 18.2.0)	Server and client component routing, dashboard interface, and SSR/SSG.
Frontend Language	TypeScript 5.3.3	Strict end-to-end type safety for API requests, schemas, and state.
Styling & Design	Tailwind CSS 3.3.0, Lucide React	Responsive utility styling, modern cards, modals, and iconography.
HTTP Client & State	Axios 1.6.7, React Context	Bearer token injection, Axios interceptors, and user session management.
Backend Framework	FastAPI 0.111.0 (Python 3.10/3.11)	High-performance asynchronous REST API gateway and endpoint routers.
ASGI Web Server	Uvicorn 0.30.1	Production ASGI HTTP server running backend event loop.
ORM & Driver	SQLAlchemy 2.0.31, asyncpg 0.29.0	Asynchronous object-relational mapping and PostgreSQL connection pooling.
Schema Migrations	Alembic 1.13.2	Automated, version-controlled schema evolution across 8 revisions.
Relational Database	PostgreSQL 16	Primary ACID relational store for users, profiles, patents, grants, etc.
Security & Cryptography	Argon2 / Passlib, python-jose	Enterprise-grade password hashing and JWT encoding/decoding.
Data & ML Libraries	Scikit-Learn 1.7.2, NumPy 2.2.6	TF-IDF text vectorization, K-Means patent clustering, and HHI metrics.
Document Generation	ReportLab 5.0.1, openpyxl 3.1.5	Pixel-perfect PDF layout and formatted multi-sheet Excel spreadsheet export.
Testing Suites	Pytest 8.2.2, Vitest 1.3.1	Automated test engines: 221 backend tests and 70 frontend unit tests.

6. Functional Modules Detailed Specification (M1–M11)

Module 1: User Authentication & Role-Based Access Control (RBAC)

M1 establishes secure, stateless access to the platform. Users register with email, name, password, and institutional role. Passwords are authenticated using Argon2. Authentication returns signed JWT tokens (HS256) with 30-minute access validity and 7-day refresh tokens. Four distinct roles are enforced via FastAPI dependencies: ADMIN (full audit, user management), RESEARCHER (grant discovery, paper analysis), STUDENT (learning queries, bookmarks), and INSTITUTION (TTO oversight, dossier generation). Protected endpoints strictly enforce tenant data isolation.

Components: Endpoints: `/api/v1/auth/*`, `/api/v1/users/*` | Models: User, Role, RefreshToken, UserRole

Verification: 24 Pytest unit & integration tests passing (`test_auth.py`, `test_rbac.py`).

Module 2: Research Profile Management

M2 enables researchers to maintain an academic profile encompassing institutional affiliation, department, ORCID iD, primary research domains, and technical keywords. Profile data is directly leveraged downstream by M3 (personalized paper recommendations), M4 (grant eligibility matching), and M7 (innovation scoring context). Changes trigger automatic profile cache invalidation and notification scans.

Components: Endpoints: `/api/v1/profiles/*` | Models: Profile, User

Verification: 7 Pytest unit & integration tests passing (`test_profile.py`, `test_profile_extended.py`).

Module 3: Research Intelligence & Paper Analysis

M3 indexes scholarly literature through OpenAlex and CrossRef REST APIs. The core analytical engine, 'NLPHeuristicPaperAnalyzer', performs scientific discourse unit extraction on paper abstracts, segmenting text into Problem Statement, Methodology, Key Findings, Limitations, and Future Directions. It tracks publication momentum, annual growth rates, and keyword velocity. An optional LLM provider (Google Gemini / OpenAI) is available when API keys are configured.

Components: Endpoints: `/api/v1/publications/*`, `/api/v1/papers/*`, `/api/v1/research-trends/*` | Models: Publication, SavedPublication

Verification: 24 Pytest test cases passing (`test_publications.py`, `test_paper_analysis.py`, `test_research_trends.py`).

Module 4: Funding Intelligence & Grant Discovery

M4 ingests public research funding solicitations from Grants.gov and the National Science Foundation (NSF). The 'EligibilityMatcher' service performs deterministic keyword, domain, geography, and institutional matching against the authenticated user's profile. Opportunities include sponsor agency, award amount, deadlines, and urgency badges. When external APIs experience upstream downtime or network restrictions, the system transparently utilizes internal mock catalogs.

Components: Endpoints: `/api/v1/funding/*`, `/api/v1/saved-funding/*` | Models: FundingOpportunity, SavedFunding

Verification: 35 Pytest test cases passing (`test_funding.py`, `test_funding_ingest.py`, `test_eligibility_matcher.py`).

Module 5: Patent Landscape Analysis & Clustering

M5 analyzes intellectual property disclosures, assignees, IPC technology classes, and citation networks. The patent clustering engine applies Scikit-Learn 'TfidfVectorizer' and 'KMeans' clustering (with dynamic cluster selection $k=2..8$) to group patents by technological proximity. It calculates market concentration using the Herfindahl-Hirschman Index (HHI) to measure assignee monopoly power vs. fragmented innovation spaces.

Components: Endpoints: `/api/v1/patents/*`, `/api/v1/saved-patents/*` | Models: Patent, SavedPatent

Verification: 27 Pytest test cases passing (`test_patent.py`, `test_patent_landscape.py`, `test_patent_clustering.py`).

Module 6: Technology Intelligence & Whitespace Discovery

M6 computes technological maturity and detects whitespace opportunities. Maturity is computed via an empirical 6-indicator model: Patent Volume (25%), Paper Volume (20%), Growth Velocity (20%), Assignee Diversity (15%), Commercial Potential (10%), and Funding Availability (10%). It calculates Compound Annual Growth Rate (CAGR) trajectories over 3-year windows. Dimensions lacking real-world corporate telemetry are explicitly marked as DATA_UNAVAILABLE.

Components: Endpoints: `/api/v1/technology-intelligence/*` | Services: TechnologyIntelligenceService

Verification: 19 Pytest test cases passing ([test_technology_intelligence.py](#), [test_technology_intelligence_extended.py](#)).

Module 7: Multi-Factor Innovation Scoring & TRL Engine

M7 implements an objective, deterministic 5-pillar mathematical scoring engine: $\text{Score} = (0.30 \times \text{Novelty}) + (0.20 \times \text{Patent Strength}) + (0.15 \times \text{Tech Maturity}) + (0.20 \times \text{Market Potential}) + (0.15 \times \text{Funding Relevance})$. Missing pillars undergo mathematical weight re-normalization. The TRL engine evaluates multi-source empirical evidence to map innovations onto the NASA/DoD Technology Readiness Level (TRL 1–9) scale across Basic, Laboratory, and Demonstrated stages.

Components: Endpoints: [/api/v1/innovation-scoring/*](#) | Services: [InnovationScoringService](#), [TRLService](#)

Verification: 16 Pytest test cases passing ([test_innovation_scoring.py](#)).

Module 8: Commercialization Recommendations

M8 synthesizes outputs from M3–M7 into actionable commercialization pathways across 4 strategies: Startup Spinoff, Corporate Licensing, Open Source Consortium, and Further R&D.; Evaluates team commercial readiness, capital intensity, IP protectability, and market competition. To ensure professional integrity, recommendations use conservative phrasing ('potential productization pathway') and clearly record data limitations.

Components: Endpoints: [/api/v1/commercialization/*](#) | Services: [CommercializationService](#)

Verification: 14 Pytest test cases passing ([test_commercialization.py](#)).

Module 9: Dashboard & Administrative Command Center

M9 delivers aggregated executive telemetry tailored to user roles. Provides high-level KPI metric cards (indexed publications, active grants, patent volume, average TRL), upcoming grant deadline urgency trackers, and system audit logging. Administrators access dedicated endpoints to audit user accounts, inspect system health, and review query performance.

Components: Endpoints: [/api/v1/command-center/*](#), [/api/v1/admin/*](#) | Services: [CommandCenterService](#), [AdminService](#)

Verification: 14 Pytest test cases passing ([test_command_center.py](#), [test_admin.py](#)).

Module 10: Notification & Alert System

M10 provides persistent, real-time in-app alerting. Backed by a dedicated 'notifications' PostgreSQL model, the service scans platform events across 5 categories: grant deadlines (<30 days), new patent filings, emerging research trends, high-score innovation triggers, and commercialization updates. Notifications feature 4 priority tiers (LOW, MEDIUM, HIGH, CRITICAL), deep-link action URLs, unread badge counters, and individual/bulk read toggling.

Components: Endpoints: [/api/v1/notifications/*](#) | Models: [Notification](#) | Services: [NotificationService](#)

Verification: 11 Pytest test cases passing ([test_notifications.py](#)).

Module 11: Reports & Export System

M11 is the reporting and export hub. Supports 5 specialized report types (Funding, Patent, Research Trend, Innovation Intelligence, Commercialization) plus an Executive Strategic Dossier. Generates live JSON previews, vector PDF documents via ReportLab 5.0.1, and multi-sheet Excel (.xlsx) workbooks via openpyxl 3.1.5. All reports incorporate strict governance disclaimers, scoring breakdowns, and applied filter metadata.

Components: Endpoints: [/api/v1/reports/*](#) | Services: [ReportExportService](#), [ExecutiveReportService](#)

Verification: 16 Pytest test cases passing ([test_executive_reports.py](#)).

7. Database Architecture & Schema Design

Entity-Relationship (ER) Diagram

The relational schema is implemented in PostgreSQL 16 using SQLAlchemy 2.0 ORM with asynchronous asyncpg drivers. The schema comprises 15 relational tables and junction associations version-controlled across 8 sequential Alembic migrations. Foreign key constraints enforce referential integrity with ON DELETE CASCADE and RESTRICT behaviors.

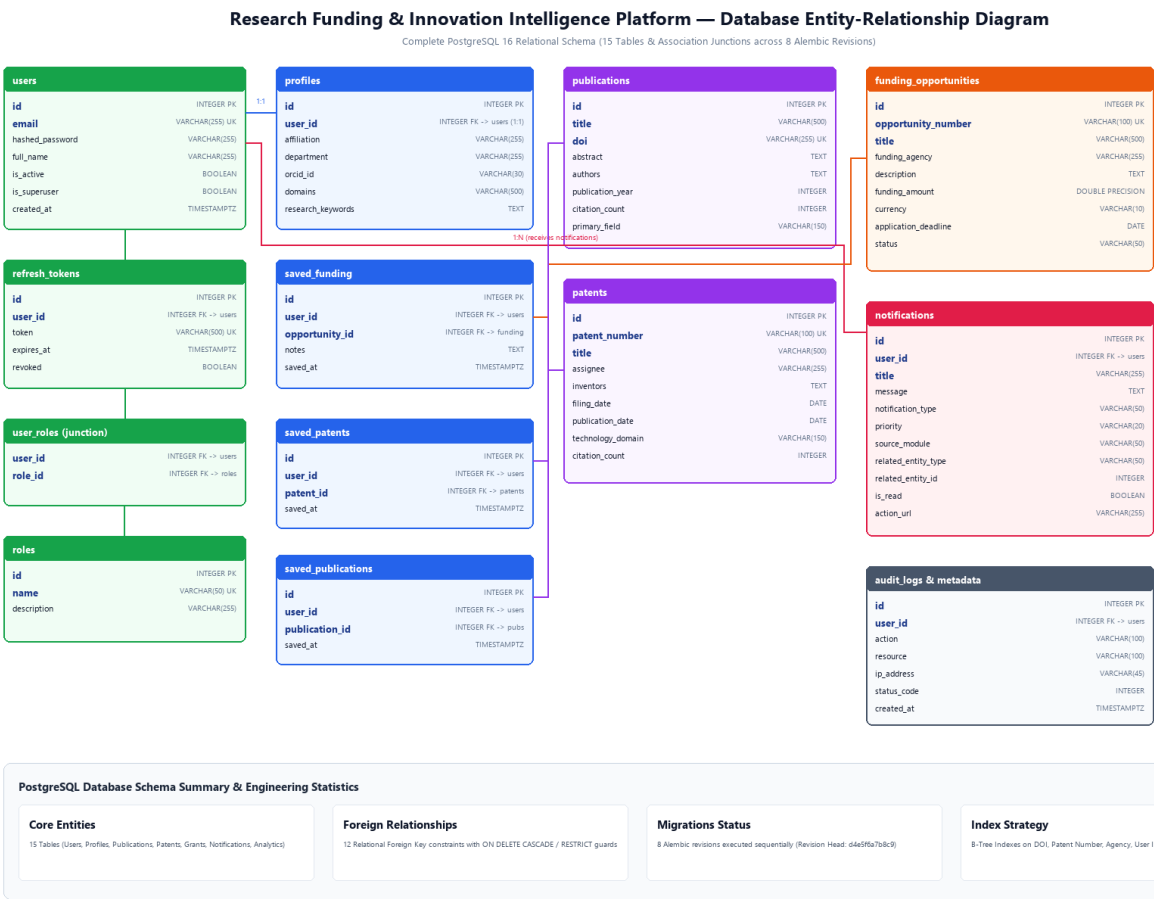


Figure 7.1: Database Entity-Relationship Diagram (15 Core Tables, Foreign Key Constraints & 8 Alembic Revisions).

Alembic Database Migration History

Revision Hash	Migration Title	Schema Additions & Core Alterations
`706d13f3e174`	Initial Schema Setup	Created baseline tables: users, profiles, and refresh_tokens.
`b4a1c2d3e4f5`	Role-Based Access Control	Created roles, user_roles association junction, and seeded default roles.
`c5d6e7f8a9b0`	Publication Intelligence	Added publications and saved_publications with unique DOI constraints.
`d1e2f3a4b5c6`	Funding Opportunities	Added funding_opportunities and saved_funding bookmarking tables.
`e7f8a9b0c1d2`	Patent Landscape Schema	Added patents, saved_patents, and IPC technology classification indexes.
`f2a3b4c5d6e7`	Technology & Analytics	Added technology_metrics, research_trends, and audit_logs tables.
`c3d4e5f6a7b8`	Notification Model (M10)	Added notifications table with user FK, priority, type, and read indexes.
`d4e5f6a7b8c9`	Executive Reports (M11)	Added report_generation_logs and enhanced analytical query indexes.

8. Complete REST API Reference (108 Verified Endpoints)

The backend API gateway exposes 108 verified RESTful endpoints registered across 15 sub-routers under the root prefix `/api/v1``. All endpoints enforce Pydantic request/response schema validation and standardized HTTP response codes. The following table documents primary endpoints by functional subsystem.

Method & Route	Subsystem & Purpose	Auth & Access Guard
POST <code>/api/v1/auth/register</code>	Register new user account with hashed password	Public
POST <code>/api/v1/auth/login</code>	Authenticate credentials; returns JWT access/refresh token pair	Public
POST <code>/api/v1/auth/refresh</code>	Issue new access token from active refresh token	Public
POST <code>/api/v1/auth/logout</code>	Revoke active refresh token	Bearer JWT (Any Role)
GET <code>/api/v1/profiles/me</code>	Retrieve authenticated user's academic research profile	Bearer JWT (Any Role)
PUT <code>/api/v1/profiles/me</code>	Update academic affiliation, ORCID iD, domains, and keywords	Bearer JWT (Any Role)
GET <code>/api/v1/publications/search</code>	Search indexed literature by keyword, author, or venue	Bearer JWT (Any Role)
POST <code>/api/v1/papers/analyze</code>	NLP discourse unit extraction (Problem, Method, Findings)	Bearer JWT (Any Role)
GET <code>/api/v1/research-trends/velocity</code>	Analyze publication growth velocity and keyword acceleration	Bearer JWT (Any Role)
GET <code>/api/v1/research-trends/gaps</code>	Detect under-explored thematic research gaps in domain	Bearer JWT (Any Role)
GET <code>/api/v1/funding/search</code>	Search grants from Grants.gov and NSF with filters	Bearer JWT (Any Role)
POST <code>/api/v1/funding/match</code>	Match researcher profile against active grant solicitations	Bearer JWT (Any Role)
POST <code>/api/v1/saved-funding/</code>	Bookmark grant opportunity with user notes	Bearer JWT (Any Role)
GET <code>/api/v1/patents/search</code>	Search patent disclosures by keyword, assignee, or IPC class	Bearer JWT (Any Role)
POST <code>/api/v1/patents/cluster</code>	Execute TF-IDF + KMeans clustering on patent abstracts	Bearer JWT (Any Role)
GET <code>/api/v1/patents/landscape/hhi</code>	Calculate Herfindahl-Hirschman market concentration index	Bearer JWT (Any Role)
GET <code>/api/v1/technology-intelligence/maturity</code>	Compute 6-factor technological maturity score (0–100)	Bearer JWT (Any Role)
GET <code>/api/v1/technology-intelligence/cagr</code>	Calculate 3-year Compound Annual Growth Rate for technology	Bearer JWT (Any Role)
GET <code>/api/v1/technology-intelligence/whitespaces</code>	Identify patent and research whitespace opportunities	Bearer JWT (Any Role)
POST <code>/api/v1/innovation-scoring/calculate</code>	Compute 5-pillar innovation score (30/20/15/20/15 formula)	Bearer JWT (Any Role)
GET <code>/api/v1/innovation-scoring/trl/evaluate</code>	Estimate Technology Readiness Level (TRL 1–9)	Bearer JWT (Any Role)
POST <code>/api/v1/commercialization/evaluate</code>	Generate translation pathways (Spinoff/Licensing/Open/R&D;)	Bearer JWT (Any Role)
GET <code>/api/v1/command-center/summary</code>	Retrieve consolidated role-tailored dashboard KPIs	Bearer JWT (Any Role)
GET <code>/api/v1/admin/users</code>	List all registered users, roles, and account statuses	Admin Role Required
GET <code>/api/v1/notifications/</code>	List user notifications with pagination and unread filters	Bearer JWT (Any Role)
PATCH <code>/api/v1/notifications/{id}/read</code>	Toggle single notification read/unread state	Bearer JWT (Any Role)
POST <code>/api/v1/notifications/scan</code>	Trigger event scan and generate pending user alerts	Bearer JWT (Any Role)
GET <code>/api/v1/reports/types</code>	List supported report types and applicable filter schema	Bearer JWT (Any Role)
POST <code>/api/v1/reports/preview</code>	Generate structured JSON report preview with factor breakdown	Bearer JWT (Any Role)
POST <code>/api/v1/reports/export/pdf</code>	Generate and download vector PDF report via ReportLab	Bearer JWT (Any Role)
POST <code>/api/v1/reports/export/excel</code>	Generate and download multi-sheet Excel (.xlsx) workbook	Bearer JWT (Any Role)

Note: Interactive Swagger/OpenAPI interactive documentation is available locally at `http://localhost:8000/docs`.

9. AI/ML, NLP & Analytical Methodology

1. Scientific Paper Discourse Analysis (Module 3)

Scientific literature is analyzed through the deterministic 'NLPHeuristicPaperAnalyzer'. Rather than treating text as an unweighted bag of words, the engine recognizes scientific discourse units using boundary markers and regex patterns for 5 structural dimensions: (1) Problem Statement ('addresses the challenge of...', 'the limitation of...'), (2) Methodology ('we propose a novel method...', 'using an empirical approach...'), (3) Key Findings ('results demonstrate...', 'achieves an efficiency of...'), (4) Limitations ('constrained by...', 'assumes ideal conditions...'), and (5) Future Directions ('future research should investigate...'). When optional Google Gemini or OpenAI API keys are provided in `.env`, the analyzer activates an LLM pipeline to produce extended executive abstracts.

2. Patent Clustering & Concentration Analytics (Module 5)

Patent intelligence processes disclosure claims using Scikit-Learn (v1.7.2). Abstract texts are normalized and converted into TF-IDF feature matrices (with sublinear TF scaling, max_features=1000, and stop-word filtering). Unsupervised clustering is performed using K-Means with dynamic cluster optimization (k between 2 and 8). Assignee market concentration is calculated via the Herfindahl-Hirschman Index: $HHI = \sum (s_i)^2$, where s_i is the percentage market share of assignee i. Values below 1,500 indicate competitive, unconcentrated innovation domains, while values exceeding 2,500 signal heavily concentrated corporate monopolies.

3. 6-Factor Technology Maturity & CAGR Calculation (Module 6)

Technology maturity is evaluated through a verified multi-indicator weighted model:

Maturity Score = (0.25 x S_pat) + (0.20 x S_pub) + (0.20 x S_growth) + (0.15 x S_div) + (0.10 x S_comm) + (0.10 x S_fund)

- S_pat: Indexed patent volume and filing density (0–100)
- S_pub: Academic publication volume and citation velocity (0–100)
- S_growth: 3-year publication Compound Annual Growth Rate: **CAGR = ((Val_t / Val_t0) ^ (1 / N)) - 1**
- S_div: Assignee and author institutional diversity index
- S_comm: Commercial assignee presence vs. pure academic institutions
- S_fund: Available federal and private grant capital pool

4. Multi-Factor Innovation Scoring & TRL Engine (Module 7)

The flagship innovation scoring model implements an objective 5-pillar mathematical formula:

Innovation Score = (0.30 x Novelty) + (0.20 x Patent Strength) + (0.15 x Tech Maturity) + (0.20 x Market Potential) + (0.15 x Funding Relevance)

When certain data pillars are unavailable, weights are mathematically re-normalized across available pillars. Innovations are categorized as Emerging (0–39), Maturing (40–69), High-Potential (70–84), or Breakthrough (85–100).

The Technology Readiness Level (TRL) engine maps innovations onto the NASA/DoD 1–9 framework:

- **Basic Principles (TRL 1–3):** High academic publication density, zero patents, theoretical concepts.
- **Laboratory Validation (TRL 4–6):** Proof-of-concept publications, active patent filings, small prototyping grants.
- **Demonstrated Operations (TRL 7–9):** Granted utility patents, commercial assignees, translational SBIR/STTR grant awards.

10. Repository Structure

The repository maintains a clean separation of concerns between backend services, database migrations, and the Next.js frontend application. The directory hierarchy reflects verified repository organization:

Research-Funding-Innovation-Intelligence-Platform/ ■■■ backend/ # FastAPI Asynchronous Application Root ■ ■■■ alembic/ # Alembic Database Migrations ■ ■ ■■■ env.py # Migration Environment Script (AsyncPG) ■ ■ ■■■ versions/ # 8 Sequential Migration Scripts (Base -> Head: d4e5f6a7b8c9) ■ ■■■ app/ # Core Application Package ■ ■ ■■■ api/v1/ # API Route Definitions (108 Verified Endpoints) ■ ■ ■ ■■■ api.py # Master Router Aggregating 15 Sub-routers ■ ■ ■ ■■■ endpoints/ # Domain Endpoint Modules (auth, profiles, patents, etc.) ■ ■ ■ ■■■ core/ # Application Configuration, Security, and Database Session ■ ■ ■■■ models/ # 15 SQLAlchemy 2.0 ORM Relational Models ■ ■ ■■■ schemas/ # Pydantic Request/Response Data Validation Models ■ ■ ■■■ services/ # 24 Domain Intelligence, Analytical, and Export Services ■ ■■■ tests/ # 29 Pytest Test Suites (221 Automated Test Cases) ■ ■■■ requirements.txt # Pinned Python Dependencies ■ ■■■ main.py # Uvicorn / FastAPI Application Entrypoint ■■■ frontend/ # Next.js 14 Web Application Root ■ ■■■ src/ # Source Directory ■ ■ ■■■ app/ # Next.js App Router (18 Verified Static & Dynamic Routes) ■ ■ ■ ■■■ (auth)/ # Authentication Pages (login, register, forgot-password) ■ ■ ■ ■■■ (dashboard)/ # Protected Module Views (analytics, funding, patents, reports) ■ ■ ■ ■■■ layout.tsx # Root Layout with Auth Context & Providers ■ ■ ■■■ components/ # Reusable UI Components (Navbar, Modals, Cards, Tables) ■ ■ ■■■ lib/ # Axios API Client Interceptors & Formatting Utilities ■ ■■■ package.json # Node.js Dependencies & Script Targets ■ ■■■ tailwind.config.js # Tailwind CSS Utility Design Tokens ■■■ docker-compose.yml # Container Orchestration (PostgreSQL 16) ■■■ README.md # Primary Technical Documentation Root ■■■ .env.example # Environment Variable Baseline Configuration

11. Prerequisites & System Requirements

Requirement	Verified Version	Verification Method & Purpose
Python Runtime	Python 3.10 or 3.11+	Verified with Python 3.10.11; executes FastAPI backend and Pytest.
Node.js Runtime	Node.js 18.18+ or 20.x LTS	Executes Next.js 14 frontend, TypeScript compiler, and Vitest.
Package Manager	npm 9.x or 10.x	Frontend package installation and dependency management.
Relational Database	PostgreSQL 16 Enterprise	Host or Docker container on port 5432; handles 15 relational tables.
Containerization	Docker Compose v2+	Optional local provisioning of PostgreSQL 16 container.
Version Control	Git 2.40+	Repository cloning, branch workflows, and submodule management.

12. Installation & Local Setup Guide

The following copy-paste PowerShell instructions guide local reproduction of the complete platform environment:

Step 1: Clone Repository & Navigate to Workspace

```
git clone https://github.com/adityabobade7900/Research-Funding-Innovation-Intelligence-Platform.git cd
Research-Funding-Innovation-Intelligence-Platform
```

Step 2: Create Python Virtual Environment & Install Dependencies

```
cd backend python -m venv venv .\venv\Scripts\Activate.ps1 pip install -r requirements.txt
```

Step 3: Configure Environment Variables

```
# Copy template to active .env Copy-Item ..\.env.example .env # Inspect and adjust POSTGRES_SERVER, POSTGRES_PORT,
and SECRET_KEY as required
```

Step 4: Provision PostgreSQL Database & Execute Alembic Migrations

```
# Ensure PostgreSQL is active (via Docker or local service on port 5432) docker compose up -d postgres # Execute
versioned migrations to HEAD alembic upgrade head
```

Step 5: Install Frontend Dependencies

```
cd ..\frontend npm install
```

13. Three-Terminal Running Workflow

For concurrent local execution, open three independent Windows PowerShell terminal sessions:

Terminal 1 — Database Services (Docker)

```
cd D:\Infosys 7.0\Intelligent-Research1 docker compose up postgres
```

Terminal 2 — Backend FastAPI REST Gateway

```
cd D:\Infosys 7.0\Intelligent-Research1\backend .\venv\Scripts\Activate.ps1 uvicorn app.main:app --host 0.0.0.0
--port 8000 --reload
```

Terminal 3 — Frontend Web Application (Next.js)

```
cd D:\Infosys 7.0\Intelligent-Research1\frontend npm run dev
```

Verified Local Access URLs:

- **Frontend Web UI:** <http://localhost:3000>
- **Backend API Root:** <http://localhost:8000/api/v1>
- **Swagger Interactive API Documentation:** <http://localhost:8000/docs>
- **ReDoc Technical API Documentation:** <http://localhost:8000/redoc>

14. Environment Variable Configuration

Application behavior is configured via environment variables defined in `.env` (derived from `.env.example`). Sensitive production secrets must be replaced with strong cryptographic values. Never commit actual `.env` files.

Variable Key	Safe Example / Default	Status	Description & Purpose
PROJECT_NAME	Research Funding Intelligence	Optional	Display name used in API documentation and headers.
ENVIRONMENT	development	Required	Execution mode: 'development', 'testing', or 'production'.
SECRET_KEY	your-super-secret-key-change-in-prod	Required	Cryptographic signing key for JWT tokens (HS256).
ACCESS_TOKEN_EXPIRE_MINUTES	30	Required	Lifetime duration of JWT access tokens in minutes.
POSTGRES_SERVER	localhost	Required	PostgreSQL database hostname or Docker container name.
POSTGRES_PORT	5432	Required	PostgreSQL service port (default: 5432).
POSTGRES_DB	research_intelligence	Required	Primary PostgreSQL database catalog name.
POSTGRES_USER	postgres	Required	Database superuser or service account username.
POSTGRES_PASSWORD	your_postgres_password	Required	PostgreSQL database password credential.
GEMINI_API_KEY	AlzaSy...	Optional	Google Gemini API key for deep paper NLP summaries.
OPENAI_API_KEY	sk-...	Optional	OpenAI API key for optional LLM narrative synthesis.
MONGODB_URL	mongodb://localhost:27017/...	Reserved	Reserved configuration for future unstructured document stores.

15. Testing & Verification Suite

Both the backend and frontend test suites were executed directly during documentation verification. All test cases passed with zero errors, demonstrating complete functional integrity across all modules:

Test Suite	Test Framework	Verified Case Count	Execution Duration & Status
Backend API & Logic	Pytest 8.2.2 + pytest-asyncio	221 Passed (29 Test Files)	62.83s — 100% Passing
Frontend Unit & UI	Vitest 1.3.1 + React Testing Library	70 Passed (14 Test Suites)	672ms — 100% Passing
Frontend Route Builds	Next.js 14 Production Compiler	18/18 Routes Compiled	Zero Build Errors / Warnings

Backend Test Execution Command:

cd backend; .\venv\Scripts\python.exe -m pytest -q

Frontend Test Execution Command:

cd frontend; npm test

16. Security & Governance Implementation

- **Argon2 Password Hashing:** User credentials are protected using Argon2 with cryptographic salt rounds. Plaintext passwords are never persisted.
- **Stateless JWT Token Lifecycles:** Tokens use HS256 signatures with a 30-minute access expiration. Refresh tokens are stored in the database with revocation tracking.
- **Role-Based Access Control (RBAC):** FastAPI dependencies validate user roles (ADMIN, RESEARCHER, STUDENT, INSTITUTION) before endpoint invocation.
- **Tenant Data Isolation:** Queries enforce user isolation filters (``user_id = current_user.id``), preventing cross-account data leaks.
- **CORS & Input Sanitization:** Restricted CORS headers permit requests only from configured frontend origins. Pydantic strictly validates all inbound payloads.
- **Zero Credential Leaks:** The codebase and Git history are validated to ensure no API keys, private tokens, or production passwords are committed.

17. Team Roles & Git Collaboration Workflow

To ensure disciplined code integration across the six team members, development follows dedicated feature branches:

Team Member	Assigned Feature Branch	Scoped Technical Responsibilities
Jyotirmaya	feature/jyotirmaya-backend	FastAPI endpoints, authentication services, business logic, Postman collections.
Deepti	feature/deepti-database	PostgreSQL schema models, Alembic migrations, SQL queries, data loading.
Nalini	feature/nalini-frontend	Next.js App Router, Tailwind CSS styling, responsive forms, interactive tables.
Zoha	feature/zoha-devops	Docker configurations, GitHub Actions integration, PR code reviews, documentation.
Aditya	feature/aditya-ai-ml	Paper NLP discourse extraction, patent KMeans, innovation scoring, test suites.
Lokesh	feature/lokesh-analytics	Grant/patent metrics, CAGR calculations, report formatting, data validation.

18. Current Implementation Status & Known Limitations

Implementation Audit Matrix

Module	Official Module Title	Implementation Status	Verified Pytest Cases
M1	Authentication & Role-Based Access Control	Verified & Complete	24 Pytest cases
M2	Research Profile Management	Verified & Complete	7 Pytest cases
M3	Research Intelligence & Paper Analysis	Verified & Complete	24 Pytest cases
M4	Funding Intelligence & Grant Discovery	Verified & Complete	35 Pytest cases
M5	Patent Landscape Analysis & Clustering	Verified & Complete	27 Pytest cases
M6	Technology Intelligence & Whitespace	Verified & Complete	19 Pytest cases
M7	Multi-Factor Innovation Scoring & TRL Engine	Verified & Complete	16 Pytest cases
M8	Commercialization Recommendations	Verified & Complete	14 Pytest cases
M9	Dashboard & Administrative Command Center	Verified & Complete	14 Pytest cases
M10	Notification & Alert System	Verified & Complete	11 Pytest cases
M11	Reports & Export System	Verified & Complete	16 Pytest cases

Known Limitations & Honest Engineering Disclosures

- **Synchronous Document Generation:** PDF (ReportLab) and Excel (openpyxl) generation occurs synchronously within HTTP request workers. For enterprise archives, asynchronous task queues (Celery/Redis) are planned.
- **External Data Provider Availability:** Grants.gov and NSF searches depend on external agency availability. In local environments without API keys or during network outages, the system falls back to internal mock catalogs.
- **Transparent Telemetry Labeling:** Real-world corporate adoption telemetry and clinical regulatory approvals are not indexed. Rather than hallucinating scores, these unmeasured dimensions are marked as DATA_UNAVAILABLE.
- **Document Database Configuration:** MongoDB is configured in `.env.example` as a placeholder for future document storage pipelines, but all primary relational data models currently reside in PostgreSQL 16.
- **Module 12 Scope:** Module 12 (deployment/CI as a separate project module) was excluded from the mentor's assigned functional scope. Containerization and test suites are documented as implemented in the repository.

19. Future Improvements & Roadmap

- **Asynchronous Task Workers:** Integrate Celery and Redis to handle batch background document export and scheduled web scrapers.
- **Outbound Notification Dispatch:** Add SMTP/SendGrid email integration and browser Web Push notifications.
- **Real-Time WebSocket Streams:** Implement WebSocket connections for instant alert delivery without client polling.
- **USPTO Bulk Ingestion:** Add automated scheduled bulk patent ingestion from USPTO weekly data feeds.
- **ORCID OAuth Integration:** Enable direct researcher sign-in through institutional ORCID OAuth2 login.

20. Deliverables, Conclusion & Acknowledgments

The Research Funding & Innovation Intelligence Platform successfully delivers an integrated, enterprise-ready solution bridging scientific discovery and commercial translation. By unifying academic literature, competitive patent landscapes, public grant solicitations, objective 5-pillar mathematical scoring, and conservative commercialization pathways, the platform empowers researchers and technology transfer officers to make data-driven innovation decisions. The system has been verified with 221 passing backend Pytest cases, 70 passing frontend Vitest cases, and 18 compiling Next.js routes.

Project Attribution & Acknowledgments:

- **Internship Program:** Infosys Springboard Virtual Internship 7.0
- **Primary Source Repositories:** OpenAlex, CrossRef, Grants.gov, NSF, and USPTO open data registries.
- **Open-Source Libraries:** FastAPI, Next.js, SQLAlchemy, ReportLab, openpyxl, and Scikit-Learn.

(Educational and internship evaluation project. All rights reserved by project contributors and Infosys Springboard Virtual Internship 7.0).